

Genome Browser Querying the Tables

HUGEN 2073

Genomic Data Visualization & Integration

Slides adapted from Ryan L. Minster, PhD, MSIS

Learning Objectives

By the end of the session, students will be able to:

- Query tables in a relational database using SQL SELECT statements
- Query tables in the UCSC Genome Browser using SQL statements in R
- ~~Extract data from BigBed files in the UCSC Genome Browser~~

Tables

- Location and name of variants with entries in the GWAS Catalog that are within 5 kbp of *ABOTSS* and TES, the trait(s) examined in the study, the p value of the association(s), and whether the variant is a copy number variant or not.
- Database
- Table
- Region
- Fields

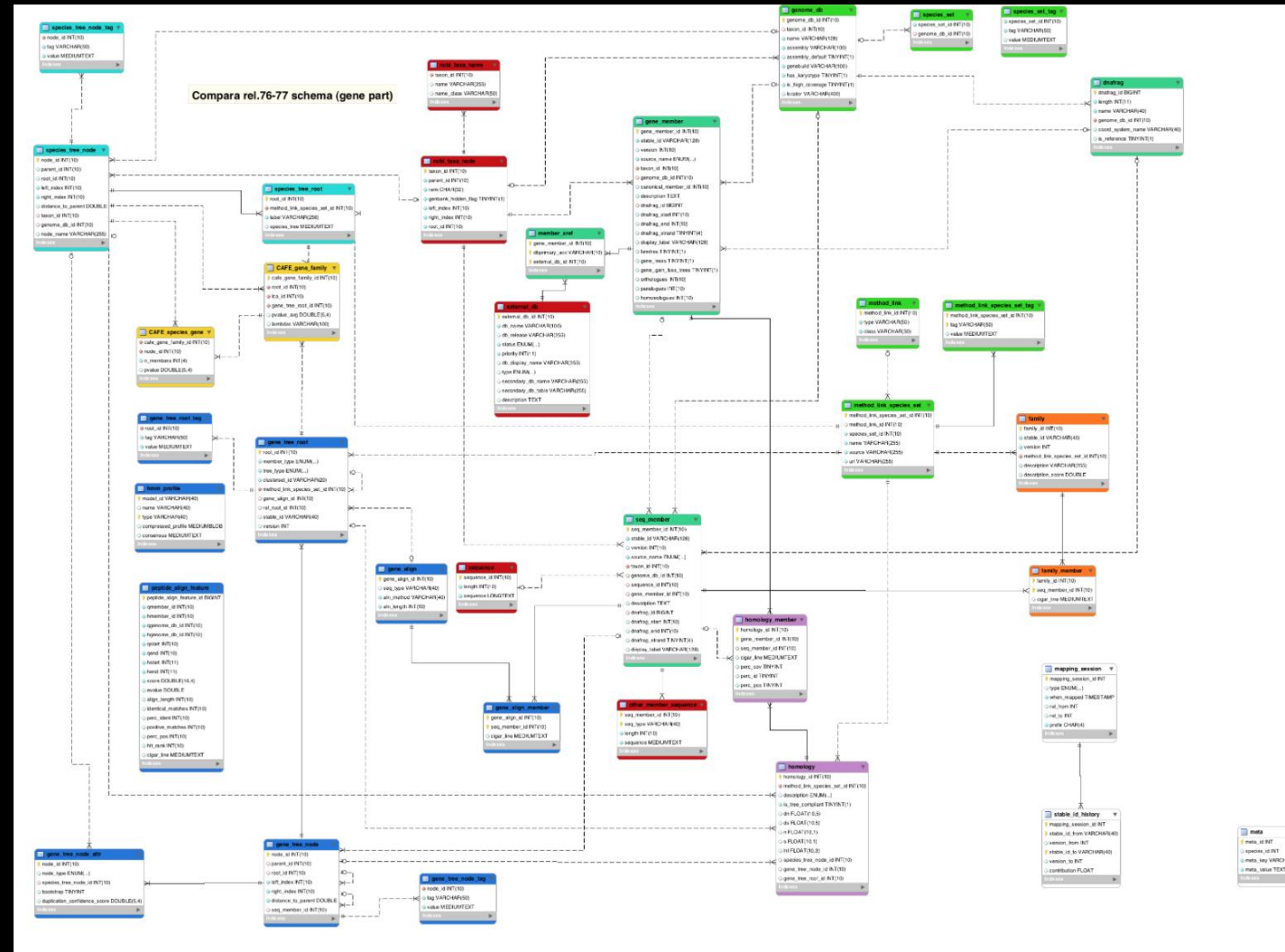
Tables

- Location and name of variants with entries among the GTEx eQTLs that are within 20 kbp of *TP53*TSS and 1 kbp of *TP53*TES, the gene symbol and tissue examined in the study, the p value of the association.
- Database
- Table
- Region
- Fields

Common Database Systems

- Flat-file: simple text files containing information in records (rows), delimited by a specific character—creating fields (columns)
 - Excel
 - Text files (.txt or .csv files)
- Relational databases: data is broken down into separate tables, each with its own set of records and fields, where the relationship between tables is given by a schema and a set of common keys
- Primary keys are columns where each value is unique in that particular table
- Foreign keys are columns in a table that are a primary key in another table.

Example of Relational Schema



uswest.ensembl.org/info/docs/api/compara/compara_schema.html

Example Relational Database

snp

chr

chrom -----> chrom
chromStart length
chromEnd
name

primary key
foreign key

Example Data

snp

chrom	chromStart	chromEnd	name
chr1	166167081	166167082	rs919033151
chr2	3000008	3000009	rs922451827
chr3	23489803	23489804	rs114587031
chr3	23489842	23489843	rs899924466
chrX	1456756	1456757	rs7055080

chr

chrom	length
chr1	248956422
chr2	242193529
chr3	198295559
chr4	190214555
chr5	181538259
chrX	156040895

SQL

- Pronounced: S-Q-L /ˌɛsˌkjuːˈɛl/ *or* sequel /ˈsiːkwəl/
- **Structured Query Language**
- Designed for **relational database management systems**
- SQL statements allow for **data queries and data manipulation**

SQL Queries

- An SQL query is built around what is called a **SELECT** statement.
- A **SELECT** statement allows you to extract (select!) data from a table, choosing particular fields and particular rows sorted into a particular order.

SELECT Statement

```
SELECT column  
  FROM table  
  WHERE row_condition  
  ORDER BY column;
```

For example:

```
SELECT chrom  
  FROM chr  
  WHERE length > 20000000;
```

SELECT Statement

For example:

```
SELECT chrom
FROM chr
WHERE length > 200000000;
```

Output:

chrom

chr1

chr2

chr	
chrom	length

chr1	248956422
chr2	242193529
chr3	198295559
chr4	190214555
chr5	181538259
chrX	156040895

SELECT Statement

For example:

```
SELECT chrom
FROM chr
WHERE length > 20000000;
```

chr	
chrom	length

chr1	248956422
chr2	242193529
chr3	198295559
chr4	190214555
chr5	181538259
chrX	156040895

Equivalent to R statement:

```
chr[chr$length > 20000000, c("chrom")] or
```

```
chr$chrom[chr$length > 20000000] or
```

```
chr %>% filter(length > 20000000) %>% select(chrom)
```

SELECT Statement multiple fields

```
SELECT column1, column2, column3, etc.  
FROM table  
WHERE row_condition  
ORDER BY column;
```

For example:

```
SELECT chrom, chromEnd, name  
FROM snp  
WHERE chromEnd > 200000000;
```

snp			
chrom	chromStart	chromEnd	name
chr1	166167081	166167082	rs919033151
chr2	30000008	30000009	rs922451827
chr3	23489803	23489804	rs114587031
chr3	23489842	23489843	rs899924466
chrX	1456756	1456757	rs7055080

I switched
tables! FYI

SELECT Statement multiple fields

For example:

```
SELECT chrom, chromEnd, name
FROM snp
WHERE chromEnd > 200000000;
```

Output:

chrom	chromEnd	name
chr1	166167082	rs919033151
chr3	23489804	rs114587031
chr3	23489843	rs899924466

snp			
chrom	chromStart	chromEnd	name

chr1	166167081	166167082	rs919033151
chr2	3000008	3000009	rs922451827
chr3	23489803	23489804	rs114587031
chr3	23489842	23489843	rs899924466
chrX	1456756	1456757	rs7055080

SELECT Statement all fields

For example:

```
SELECT *  
  FROM snp  
 WHERE chromEnd > 20000000;
```

Output:

chrom	chromStart	chromEnd	name
chr1	166167081	166167082	rs919033151
chr3	23489803	23489804	rs114587031
chr3	23489842	23489843	rs899924466

snp			
chrom	chromStart	chromEnd	name

chr1	166167081	166167082	rs919033151
chr2	3000008	3000009	rs922451827
chr3	23489803	23489804	rs114587031
chr3	23489842	23489843	rs899924466
chrX	1456756	1456757	rs7055080

SELECT Statement multiple row conditions

```
SELECT column1  
FROM table  
WHERE row_condition1 OPERATOR row_condition2 OPERATOR etc.  
ORDER BY column;
```

For example:

```
SELECT chrom, chromEnd, name  
FROM snp  
WHERE chromEnd > 200000000 AND chromEnd < 1000000000;
```

SELECT Statement multiple row conditions

For example:

```
SELECT chrom, chromEnd, name
FROM snp
WHERE chromEnd > 200000000 AND
      chromEnd < 1000000000;
```

snp			
chrom	chromStart	chromEnd	name

chr1	166167081	166167082	rs919033151
chr2	3000008	3000009	rs922451827
chr3	23489803	23489804	rs114587031
chr3	23489842	23489843	rs899924466
chrX	1456756	1456757	rs7055080

Output:

chrom	chromEnd	name
chr3	23489804	rs114587031
chr3	23489843	rs899924466

Row Logical Operators

AND logical *and* name = 'rs101' AND chrom = 'chrY'

OR logical *or* name = 'rs101' OR name = 'rs102'

NOT logical *not* chromEnd > 1000 AND NOT chrom = 'chr22'

Parenthesis (()) can be used for grouping to applied an operation order, which is otherwise left to right.

Row Conditional Operators

= equal to name = 'rs101'

<> not equal to name <> 'rs101'

> greater than chromEnd > 1000

< less than chromEnd < 1000

>= equal to chromEnd >= 1000

<= equal to chromEnd <= 1000

Row Conditional Operators

[NOT] BETWEEN [SYMMETRIC]

- inclusive range · chromEnd **BETWEEN** 1000 AND 2000
- inclusive range · chromEnd **BETWEEN SYMMETRIC** 2000 AND 1000
SYMMETRIC allows A and B to be reversed by the SQL parser if $A > B$

[NOT] LIKE

- wildcard matching · name **LIKE** 'rs10%'
- wildcard matching · name **NOT LIKE** 'rs10__'
LIKE has two wildcard operators: % and _, equivalent to regex * and .

Row Conditional Operators

[NOT] IN present in set name IN ('rs101', 'rs102')

IS [NOT] NULL is missing name IS NULL

IS [NOT] TRUE / FALSE true or false name IS FALSE

SELECT Statement if no WHERE condition

For example:

```
SELECT chrom, chromStart, chromEnd, name  
FROM snp;
```

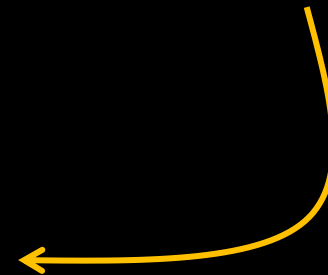
Output:

chrom	chromStart	chromEnd	name
chr1	166167081	166167082	rs919033151
chr2	3000008	3000009	rs922451827
chr3	23489803	23489804	rs114587031
chr3	23489842	23489843	rs899924466
chrX	1456756	1456757	rs7055080

snp			
chrom	chromStart	chromEnd	name

chr1	166167081	166167082	rs919033151
chr2	3000008	3000009	rs922451827
chr3	23489803	23489804	rs114587031
chr3	23489842	23489843	rs899924466
chrX	1456756	1456757	rs7055080

All rows returned.



SELECT Statement ORDER BY

For example:

```
SELECT chrom, chromEnd, name
FROM snp
WHERE chromEnd > 20000000 AND
      chromEnd < 100000000
ORDER BY chrom, chromEnd DESC;
```

Output:

chrom	chromEnd	name
chr3	23489843	rs899924466
chr3	23489804	rs114587031

snp			
chrom	chromStart	chromEnd	name

chr1	166167081	166167082	rs919033151
chr2	3000008	3000009	rs922451827
chr3	23489803	23489804	rs114587031
chr3	23489842	23489843	rs899924466
chrX	1456756	1456757	rs7055080

SELECT Statement ORDER BY

For example:

```
SELECT chrom, chromEnd, name
FROM snp
WHERE chromEnd > 200000000 AND
      chromEnd < 1000000000
ORDER BY chrom, chromEnd [ASC];
```

snp			
chrom	chromStart	chromEnd	name

chr1	166167081	166167082	rs919033151
chr2	3000008	3000009	rs922451827
chr3	23489803	23489804	rs114587031
chr3	23489842	23489843	rs899924466
chrX	1456756	1456757	rs7055080

Output:

chrom	chromEnd	name
chr3	23489804	rs114587031
chr3	23489843	rs899924466

SELECT Statement Joining tables

For example:

```
SELECT snp.chrom, snp.name, chr.length
FROM snp
      JOIN chr ON snp.chrom = chr.chrom
WHERE snp.chromEnd > 20000000 AND
      snp.chromEnd < 100000000
ORDER BY snp.chrom, snp.chromEnd;
```

Output:

```
snp.chrom  chr.length  snp.name
chr3       198295559  rs114587031
chr3       198295559  rs899924466
```

snp			
chrom	chromStart	chromEnd	name

chr1	166167081	166167082	rs919033151
chr2	3000008	3000009	rs922451827
chr3	23489803	23489804	rs114587031
chr3	23489842	23489843	rs899924466
chrX	1456756	1456757	rs7055080

chr	
chrom	length

chr1	248956422
chr2	242193529
chr3	198295559
chr4	190214555
chr5	181538259
chrX	156040895

JOIN Types

[INNER] JOIN Returns only records that have matching values in both tables

LEFT [OUTER] JOIN Returns all records from left table, and only matching records from the right table

RIGHT [OUTER] JOIN Returns all records from right table, and only matching records from the left table

FULL [OUTER] JOIN Returns all records from both tables

JOIN Types Examples

a

id name

1 John

2 María

```
SELECT a.id, a.name, b.month,  
       b.year
```

```
FROM a
```

```
JOIN b ON a.id = b.id;
```

b

id month year

1 March 1965

3 May 1964

a.id	a.name	b.month	b.year
1	John	March	1965

JOIN Types Examples

a

id name

1 John

2 María

```
SELECT a.id, a.name, b.month,  
       b.year
```

```
FROM a
```

```
LEFT JOIN b ON a.id = b.id;
```

b

id month year

1 March 1965

3 May 1964

a.id	a.name	b.month	b.year
------	--------	---------	--------

1	John	March	1965
---	------	-------	------

2	María	NULL	NULL
---	-------	------	------

JOIN Types Examples

a

id name

1 John

2 María

```
SELECT a.id, a.name, b.month,  
       b.year
```

```
FROM a
```

```
RIGHT JOIN b ON a.id = b.id;
```

b

id month year

1 March 1965

3 May 1964

a.id	a.name	b.month	b.year
------	--------	---------	--------

1	John	March	1965
---	------	-------	------

3	NULL	May	1964
---	------	-----	------

JOIN Types Examples

a

id	name
1	John
2	María

SELECT a.id, a.name, b.month,
b.year
FROM a
JOIN b ON a.id = b.id;

b

id	month	year
1	March	1965
3	May	1964

a.id	a.name	b.month	b.year
1	John	March	1965
2	María	NULL	NULL
3	NULL	May	1964

Example Query from Table Browser

```
SELECT chrom, chromStart, chromEnd, name, refNCBI  
FROM hg38.snp151  
WHERE chrom = 'chr17'  
      AND chromStart >= 7571720  
      AND chromEnd <= 7590868  
ORDER BY chrom, chromStart;
```

Output

	chrom	chromStart	chromEnd	name	refNCBI
1	chr17	7571723	7571724	rs139336280	C
2	chr17	7571725	7571726	rs1443355678	G
3	chr17	7571728	7571729	rs892233376	G
4	chr17	7571735	7571736	rs1429990273	T
5	chr17	7571737	7571738	rs1320917854	G
6	chr17	7571741	7571742	rs570627179	A
.					
.					
.					

Example JOIN Query from Table Browser

```
SELECT hg38.snp150.name,      hg38.snp150.chrom,  
       hg38.snp150.chromEnd, hg38.gwasCatalog.genes  
  
FROM hg38.snp150 JOIN hg38.gwasCatalog  
     ON hg38.snp150.name = hg38.gwasCatalog.name  
  
WHERE hg38.gwasCatalog.trait = 'Pancreatic cancer' AND  
       hg38.snp150.func = 'missense'  
  
ORDER BY hg38.snp150.chrom, hg38.snp150.chromEnd;
```

Output

	name	genes	chrom	chromEnd
1	rs146087049		chr12	113165885
2	rs34309238	PKN1	chr19	14464085
3	rs150101296		chr19	32628836
4	rs183117027	APOB	chr2	21004340
5	rs138585571		chr3	98469736
6	rs2242241	DOK2	chr8	21909370
7	rs141406935		chr9	135697569
.				
.				
.				

gwasCatalog does not
always have a genes entry
for each association
reported

SQL is not sophisticated
enough to sort this
'correctly'

Using R to Query UCSC Genome Browser

```
library(RMySQL)

mychannel <- dbConnect(MySQL(),
  user="genome",
  host="genome-mysql.cse.ucsc.edu")

data <- dbGetQuery(mychannel,
  "SELECT chromEnd FROM hg38.snp151 WHERE name = 'rs3';")

dbDisconnect(mychannel)
```

Example Code Genes near GWAS Peak

```
peakSNP <- GWAS %>%  
  filter(p == min(p)) %>%  
  filter(row_number() == 1) %>%  
  select(chr, pos)      # let's say chr = chr19 and pos = 44908822  
  
upstream <- pos - 10000  
downstream <- pos + 10000 # Get position 10 kb up- and downstream
```

Example Code Genes near GWAS Peak

```
sql <- paste0("SELECT name2 ",  
              "FROM hg38.wgEncodeGencodeBasicV39 ",  
              "WHERE (chrom = '", chr, "' AND ",  
                    "strand = '+' AND ",  
                    "txEnd > ", upstream, " AND ",  
                    "txStart < ", downstream, ") OR ",  
              "(chrom = '", chr, "' AND ",  
                    "strand = '-' AND ",  
                    "txStart > ", upstream, " AND ",  
                    "txEnd < ", pos + 10000, ");")
```

```
data <- dbGetQuery(mychannel, sql) %>% distinct()
```

Example Code Genes near GWAS Peak

```
> data
```

It's a data frame!

	name2
1	TOMM40
2	APOE
3	ENSG00000280087
4	APOC1

Soon to be Out of Date! :(

- Some of the tables in the UCSC Genome Browser have gotten *so* large that it is not longer practical (from their standpoint) to store those data in SQL tables
 - **dbSnp153 has 702 million records**
- If you look at a table schema and see:
Big Bed File: /gbdb/hg38/x/x.bb
- Then you need to use a Unix utility called **bigBedToBed** that can be downloaded from <http://hgdownload.soe.ucsc.edu/admin/exe/>